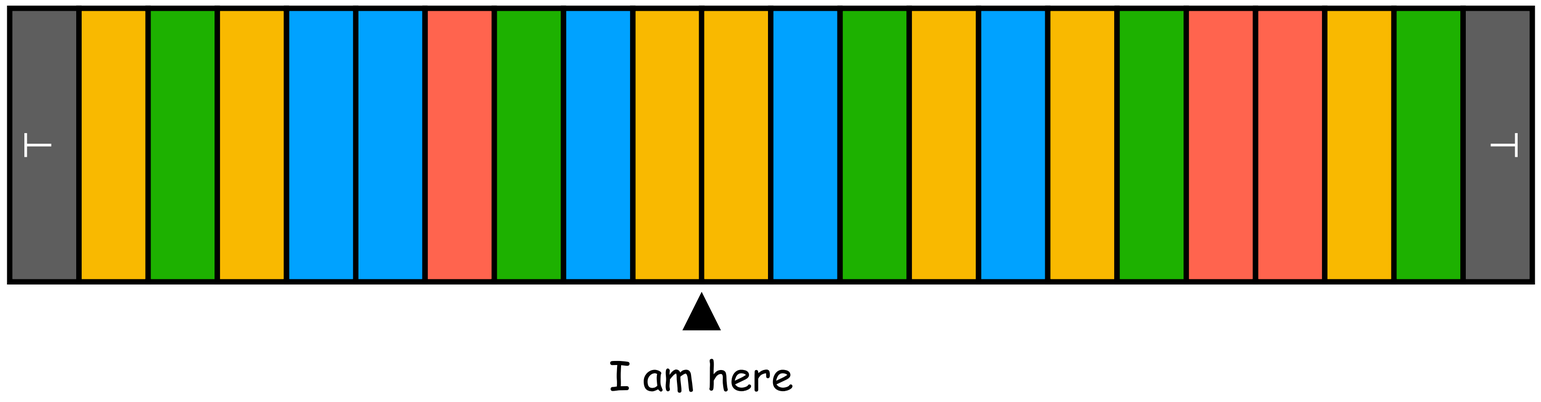
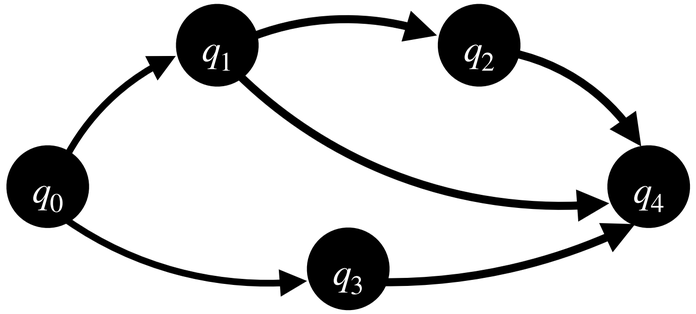
Theory of Computation

Lecture 10

What is this machine?



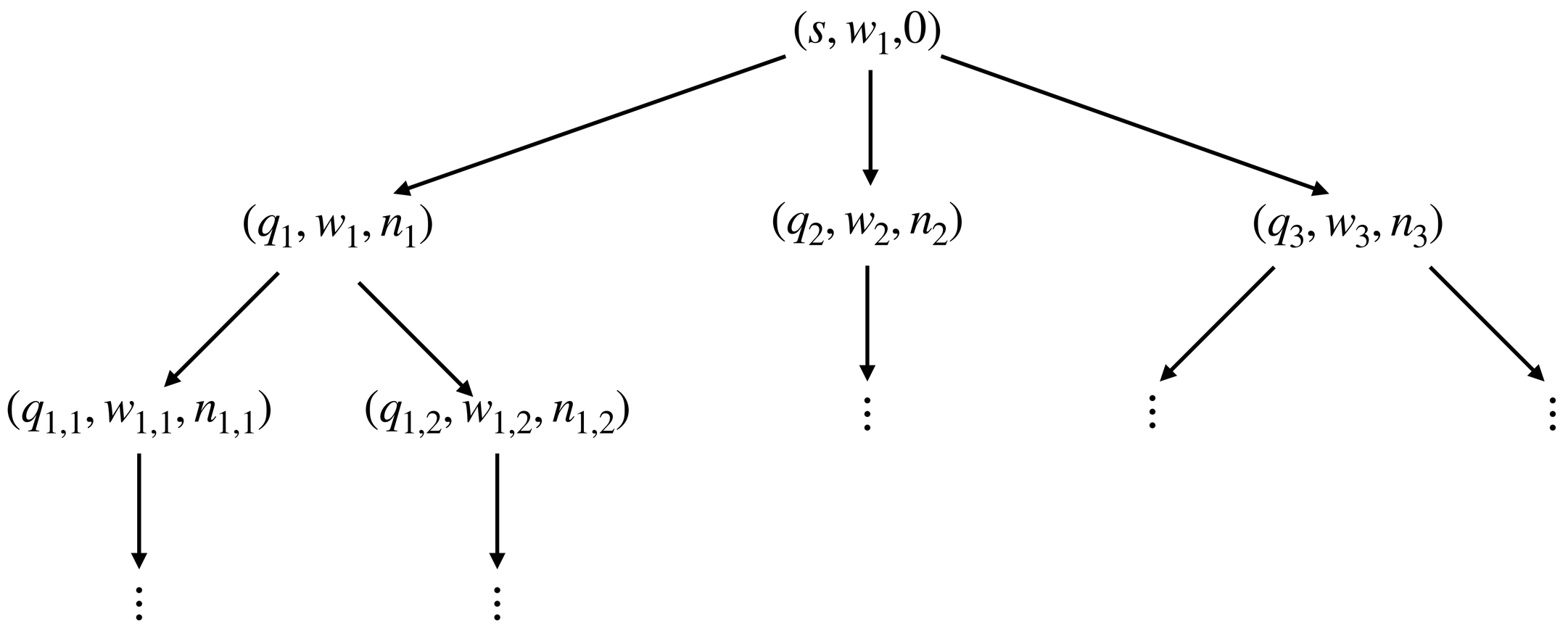


There are finitely many instructions.

There is infinite memory.

Turing Machine

Non-deterministic Turing Machines



$$O\left(k^{T(|w|)}\right)$$

Every recursive program can be simulated by a Turing Machine:

A program P consists of:

1. A finite set of function names F
2. For each function $f \in F$:
   - A finite set of instruction labels L_f
   - A distinguished start label $\ell_{f,0}$
   - A transition function describing what each instruction does

```
let rec factorial n =
  if n = 0 then 1
  else n * factorial (n - 1)
```

Every recursive program can be simulated by a Turing Machine:

Let $L = \bigcup_{f \in F} L_f$

The program counter is an element:

$\mathrm{pc} \in L$ that indicates which labeled instruction is currently being executed.

```
let rec factorial n =
  if n = 0 then 1
  else n * factorial (n - 1)
```

Every recursive program can be simulated by a Turing Machine:

Each stack frame is a triple:

(f, pc, ρ), where,

1. $f \in F$ is the current function
2. $\mathrm{pc} \in L$ is the current instruction
3. ρ (local environment) is a map of variable to values.

```
let rec factorial n =
  if n = 0 then 1
  else n * factorial (n - 1)
```

Every recursive program can be simulated by a Turing Machine:

A configuration is a (finite) stack of stack frames.

An execution is defined by a transition relation over stack frames, determined by the top stack frame.

```
let rec factorial n =
  if n = 0 then 1
  else n * factorial (n - 1)
```

Every recursive program can be simulated by a Turing Machine:

Suppose the top stack frame was

$$(f, (\mathrm{inst}; \mathrm{pc}'), \rho).$$

And the instruction $\mathrm{inst} = x \leftarrow e$.

Then evaluate the expression e, update environment ρ, and replace the frame with $\left(f, \mathrm{pc}', \rho\left[x \leftarrow v_e\right]\right)$.

```
let rec factorial n =
  if n = 0 then 1
  else n * factorial (n - 1)
```

Every recursive program can be simulated by a Turing Machine:

Suppose the top stack frame was

$$(f, (\mathrm{inst}; \mathrm{pc}'), \rho).$$

And the instruction

$\mathrm{inst} =$ if b then pc_1 else pc_2.

Then evaluate the value of b in ρ, and replace the frame with $\left(f, \mathrm{pc}_i, \rho\right)$.

```
let rec factorial n =
  if n = 0 then 1
  else n * factorial (n - 1)
```

Every recursive program can be simulated by a Turing Machine:

Suppose the top stack frame was

$$(f, (\text{inst}; \text{pc}'), \rho).$$

And the instruction

$$\text{inst} = \text{call } g(\text{args}) \to \text{pc}'.$$

Compute args in ρ, update frame to (f, pc', ρ), and push $(g, \ell_{g,0}, \rho_g)$.

```
let rec factorial n =
  if n = 0 then 1
  else n * factorial (n - 1)
```

Every recursive program can be simulated by a Turing Machine:

We can encode each configuration (f, pc, ρ) as a string.

The Turing Machine can:

1. Store configurations on the tape.
2. Locate the top frame, and read it.
3. Apply the transition rule and rewrites the encoded configuration.

The transition function of the recursive program is computable.

Every recursive program can be simulated by a Turing Machine:

Similarly, a program can simulate a Turing Machine!

Every recursive program can be simulated by a Turing Machine:

Similarly, a program can simulate a Turing Machine!

Can a Turing Machine simulate itself?

Today's Question

Is $\mathrm{RE}(\Sigma) = \mathrm{coRE}(\Sigma)$?

That is, are recursively enumerable languages closed under complement?

Languages

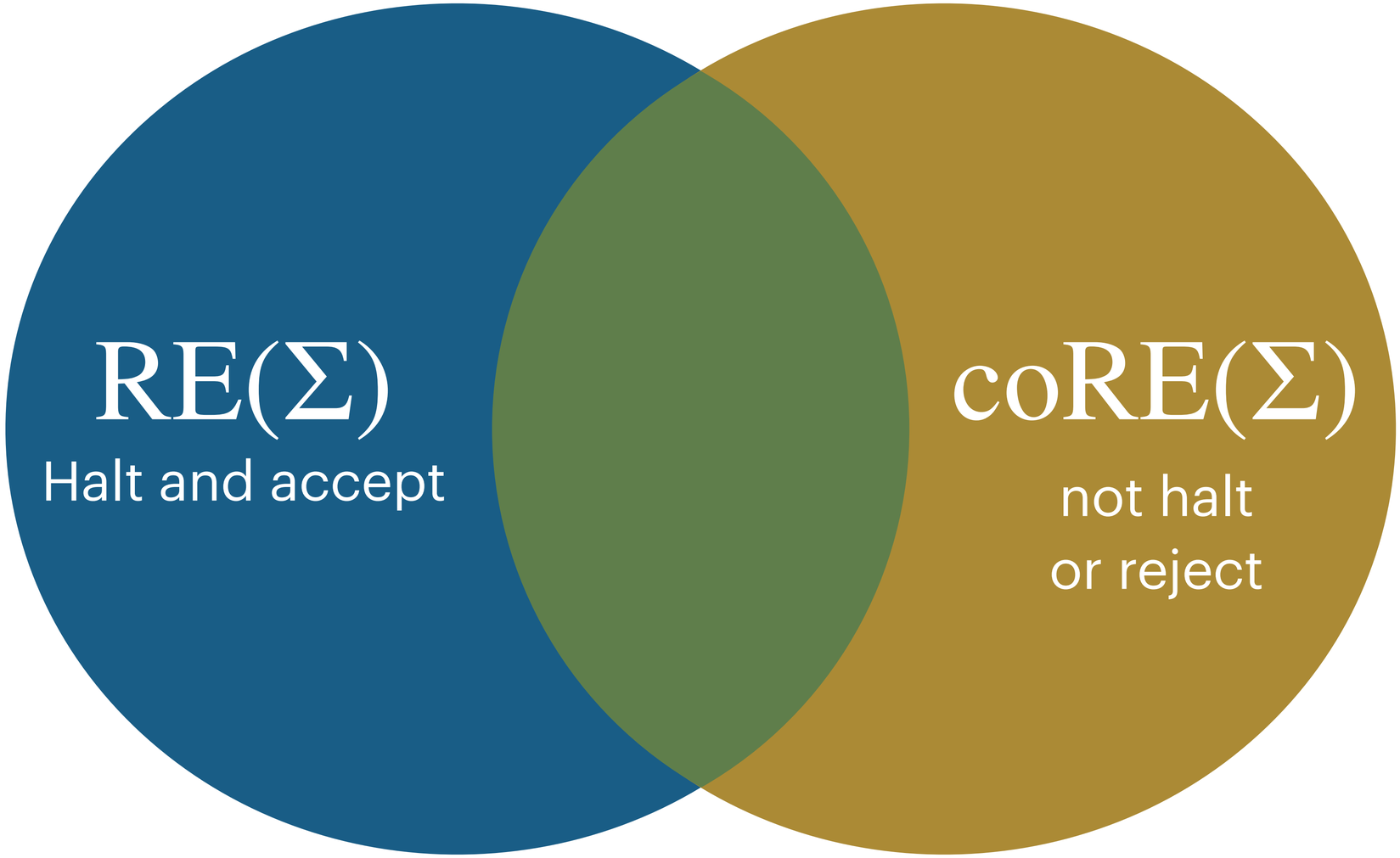
The set of words accepted by a Turing Machine is called the language of the machine.

$$\mathscr{L}(M) = \{w \in \Sigma^* : \exists w', n \,.\, (s, w, 0) \xrightarrow{*} (t, w', n)\}$$

A language $L \subseteq \Sigma^*$ is **recursively enumerable** if there exists a Turing Machine M such that $L = \mathscr{L}(M)$.

Its complement is **co-recursively enumerable**.

Today's Question



Are they equal?

We can encode the description of the machine in a string with prefix:

$$0^n 1 0^m 1 0^k 1 0^s 1 0^t 1 0^r 1 0^u 1 0^v 1$$

To mean that there are n states, m tape symbols (of which first k are input symbols, the start, accept, and reject states are s, t, and r, and the endmarker and blank symbols are u and v.

We can encode the description of the machine in a string with prefix:

$$0^n 1 0^m 1 0^k 1 0^s 1 0^t 1 0^r 1 0^u 1 0^v 1$$

To mean that there are n states, m tape symbols (of which first k are input symbols, the start, accept, and reject states are s, t, and r, and the endmarker and blank symbols are u and v.

You can then use a string like $0^p 1 0^a 1 0^q 1 0^b 1 0$ to indicate $\delta(p, a) = (q, b, L)$.

Universal Turing Machine

We can then construct a universal Turing machine U such that

$$\mathscr{L}(U) = \{M\#x : x \in \mathscr{L}(M)\}$$

Universal Turing Machine

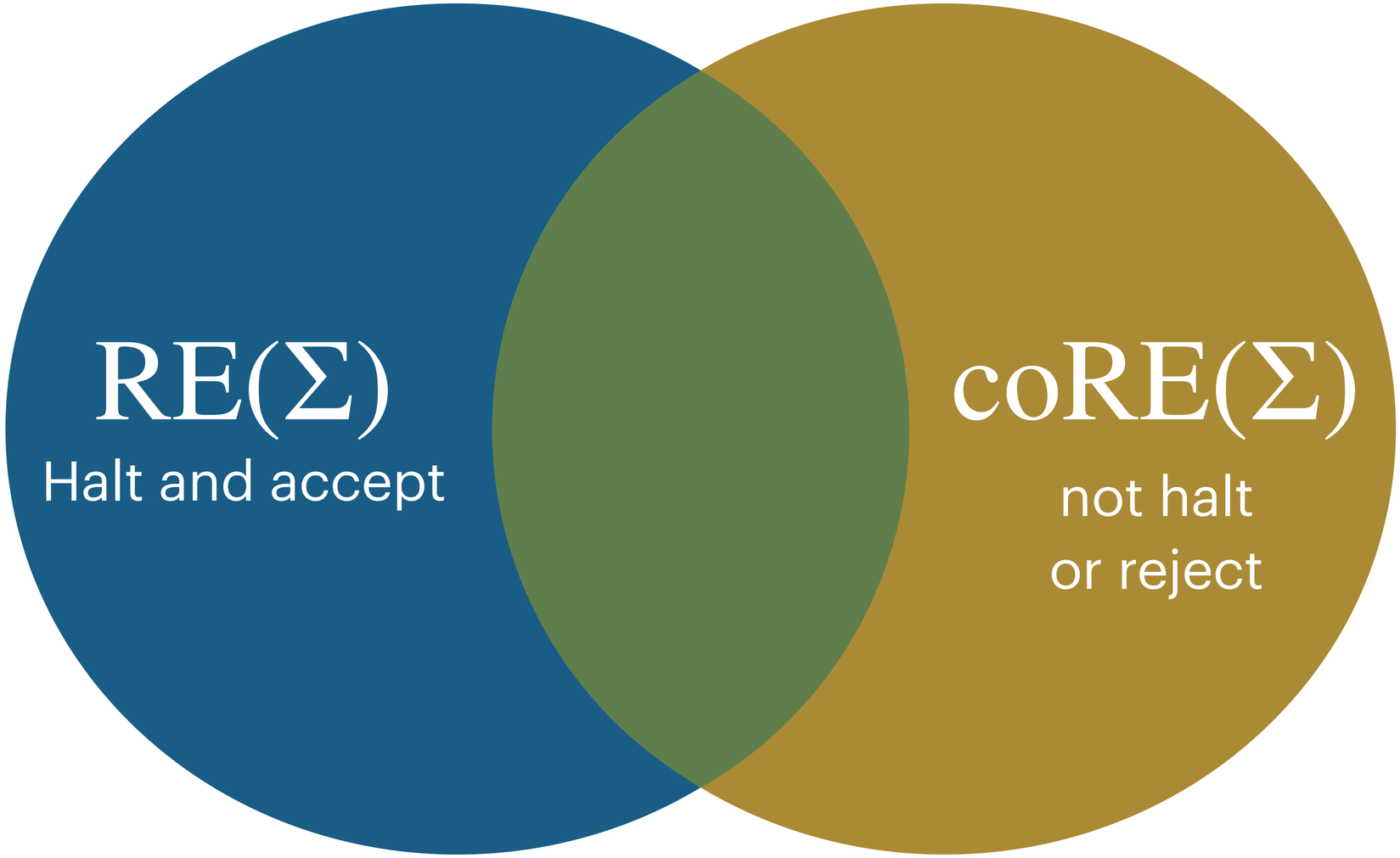
We can then construct a universal Turing machine U such that

$$\mathscr{L}(U) = \{M\#x : x \in \mathscr{L}(M)\}$$

1. The machine U first checks its input $M\#x$ is syntactically valid.
2. Then, it simulates M step-by-step.

   (a) The tape of U can be partitioned into three tracks: one for the description of M, one for tape of M, and one for the current state of and the current head position of M.

Today's Question



Is $\mathscr{L}(U) \in \mathrm{RE}(\Sigma)$? Is $\mathscr{L}(U) \in \mathrm{coRE}(\Sigma)$?

Universal Turing Machine

We can then construct a universal Turing machine U such that

$$\mathscr{L}(U) = \{M\#x : x \in \mathscr{L}(M)\}$$

$$A_{\mathrm{TM}} = \{(\langle M \rangle, w) : M \text{ accepts } w\}$$

Universal Turing Machine

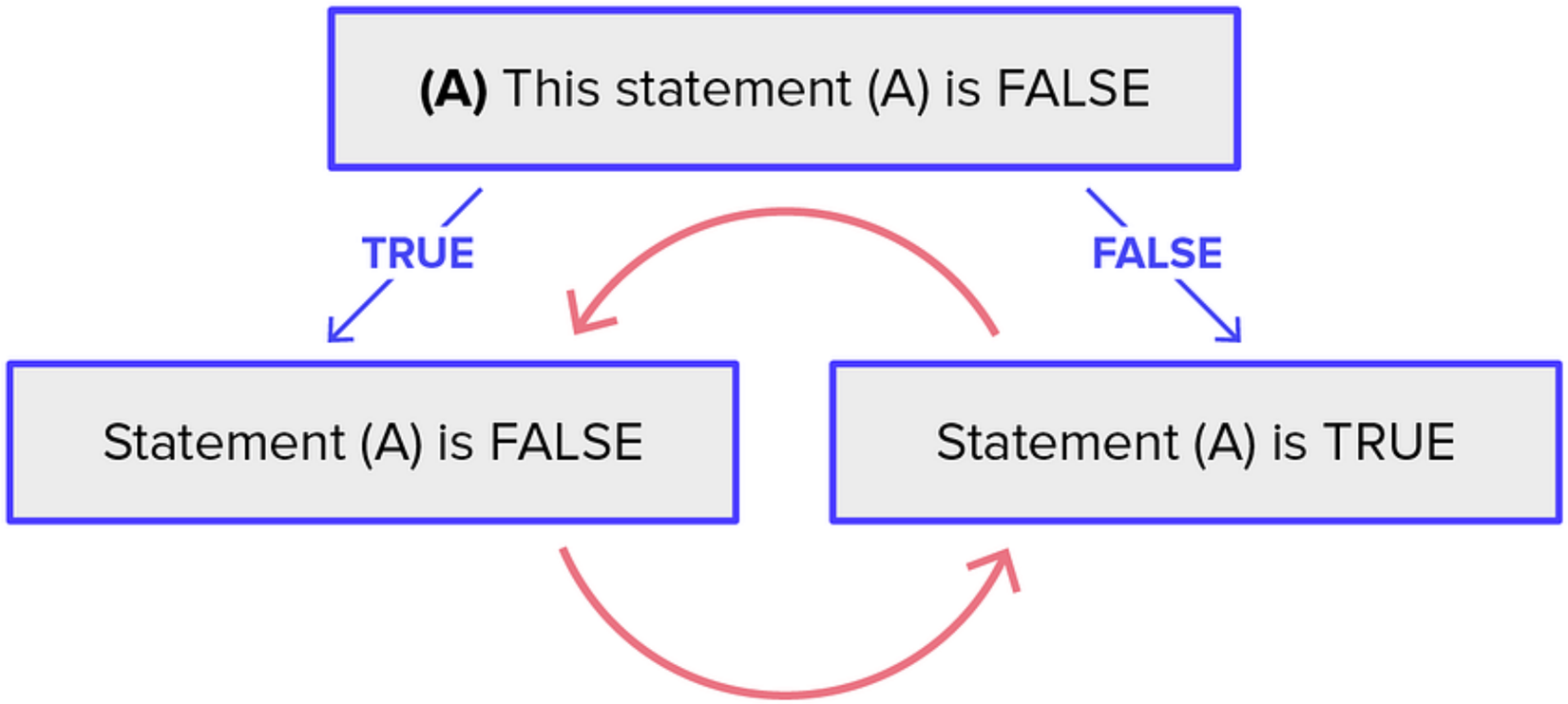
We can then construct a universal Turing machine U such that

$$\mathscr{L}(U) = \{M\#x : x \in \mathscr{L}(M)\}$$

$$A_{\mathrm{TM}} = \{(\langle M \rangle, w) : M \text{ accepts } w\}$$

Clearly, $A_{\mathrm{TM}} \in \mathrm{RE}(\Sigma)$.

So, is $\overline{A_{\mathrm{TM}}} \in \mathrm{RE}(\Sigma)$?



Pinocchio says my nose will grow.

There is no surjective function from a set to its power-set.

For sake of contradiction, let $f : S \to \mathscr{P}(S)$ be surjective. That is:

$$\forall T \in \mathscr{P}(S), \exists t \in S \text{ such that } f(t) = T.$$

There is no surjective function from a set to its power-set.

For sake of contradiction, let $f : S \to \mathscr{P}(S)$ be surjective. That is:

$$\forall T \subseteq S, \exists t \in S \text{ such that } f(t) = T.$$

There is no surjective function from a set to its power-set.

For sake of contradiction, let $f : S \to \mathscr{P}(S)$ be surjective. That is:

$$\forall T \subseteq S, \exists t \in S \text{ such that } f(t) = T.$$

Then consider the set $D = \{t \in S : t \notin f(t)\}$.

There is no surjective function from a set to its power-set.

For sake of contradiction, let $f : S \to \mathscr{P}(S)$ be surjective. That is:

$\forall T \subseteq S, \exists t \in S$ such that $f(t) = T$.

Then consider the set $D = \{t \in S : t \notin f(t)\}$.

Clearly, $D \subseteq S$.

There is no surjective function from a set to its power-set.

For sake of contradiction, let $f : S \to \mathscr{P}(S)$ be surjective. That is:

$$\forall T \subseteq S, \exists t \in S \text{ such that } f(t) = T.$$

Then consider the set $D = \{t \in S : t \notin f(t)\}$.

There exists $d \in S$ such that $f(d) = D$.

There is no surjective function from a set to its power-set.

For sake of contradiction, let $f : S \to \mathscr{P}(S)$ be surjective. That is:

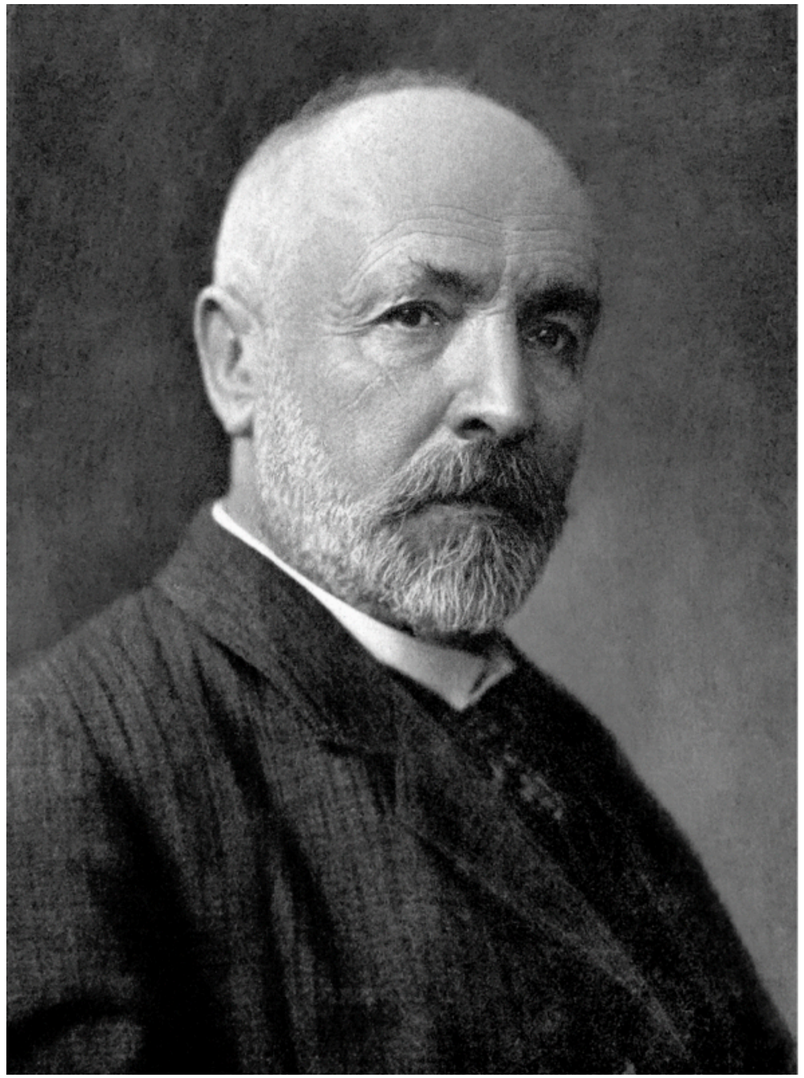
$\forall T \subseteq S, \exists t \in S$ such that $f(t) = T$.

Then consider the set $D = \{t \in S : t \notin f(t)\}$.

There exists $d \in S$ such that $f(d) = D$.

Is $d \in D$?

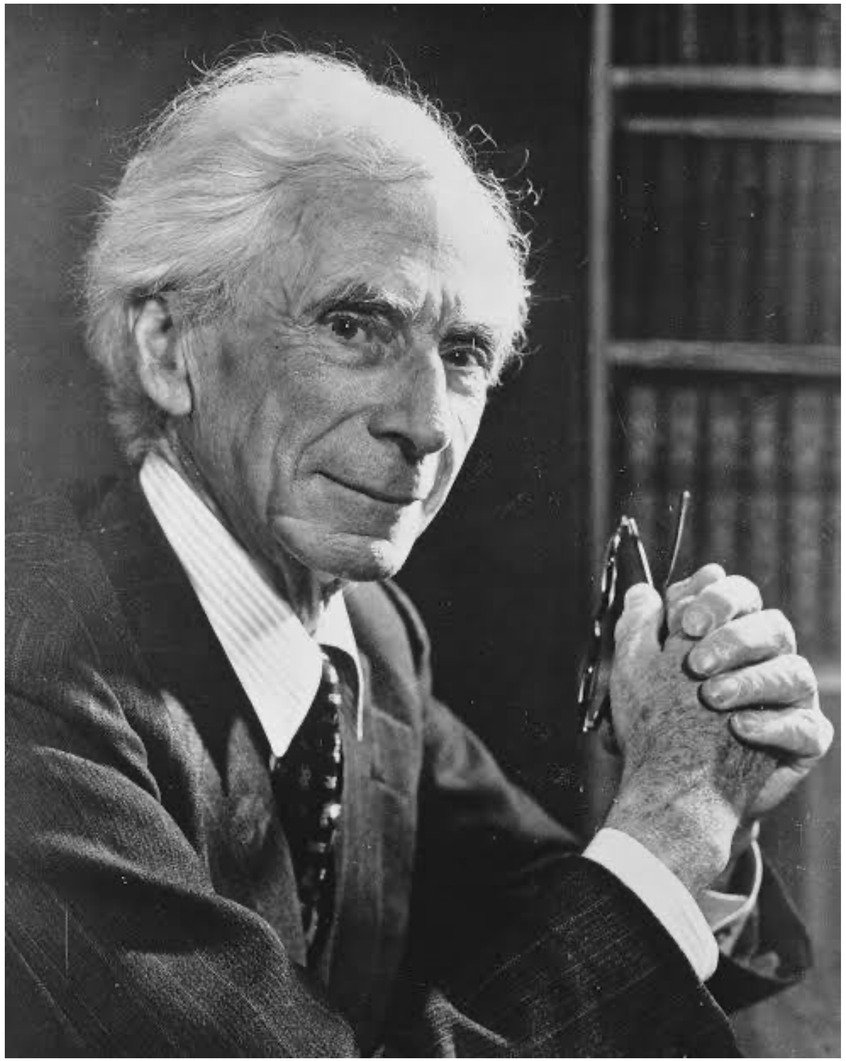
There is no surjective function from a set to its power-set.



In particular, there is no surjection from $\mathbb{N}$ to $\mathscr{P}(\mathbb{N})$.

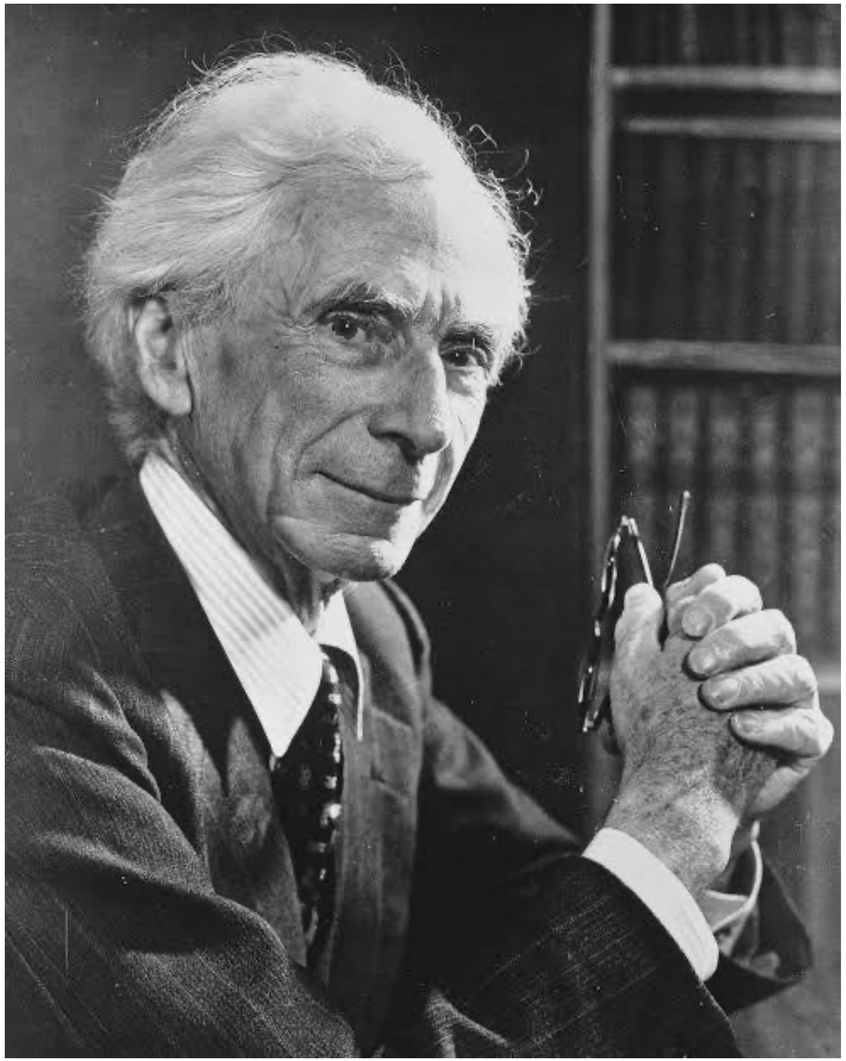
Francis Bacon wrote Shakespeare's plays.

There is no surjective function from a set to its power-set.



Instead of the set of all subsets of S, what if we consider the set of all sets?

There is no surjective function from a set to its power-set.



Instead of the set of all subsets of S, what if we consider the set of all sets?

Let $R = \{S : S \notin S\}$. Is $R \in R$?

$$
\begin{tikzcd}
T \times T \arrow[r, "f"] & Y \arrow[d, "\alpha"] \\
T \arrow[u, "\Delta"] \arrow[r, "g"'] & Y
\end{tikzcd}
$$

Consider T be a set of *objects*, and Y to be a set of *values*.

Let f evaluate object $t \in T$ on input $x \in T$.

Then α negates the output of $f(t, x)$.

$T =$ Set of well-formed sentences

$$\begin{tikzcd}
T \times T \arrow[r, "f"] & \{0,1\} \arrow[d, "\neg"] \\
T \arrow[u, "\Delta"] \arrow[r, "g"'] & \{0,1\}
\end{tikzcd}$$

Consider T be a set of *objects*, and Y to be a set of *values*.

Let f evaluate object $t \in T$ on input $x \in T$.

Then α negates the output of $f(t, x)$.

$T =$ Set of well-formed sentences

$$\begin{tikzcd}
T \times T \arrow[r, "f"] & \{0,1\} \arrow[d, "\neg"] \\
T \arrow[u, "\Delta"] \arrow[r, "g"'] & \{0,1\}
\end{tikzcd}$$

$f(t, x)$ = "sentence t is true when talking about x"

$T =$ Set of well-formed sentences

$$\begin{tikzcd}
T \times T \arrow[r, "f"] & \{0,1\} \arrow[d, "\neg"] \\
T \arrow[u, "\Delta"] \arrow[r, "g"'] & \{0,1\}
\end{tikzcd}$$

$f(\Delta(t))$ = "sentence t is true when talking about t"

$T =$ Set of well-formed sentences

$$\begin{tikzcd} T \times T \arrow[r, "f"] & \{0,1\} \arrow[d, "\neg"] \\ T \arrow[u, "\Delta"] \arrow[r, "g"'] & \{0,1\} \end{tikzcd}$$

$\alpha\left(f(\Delta(t))\right)$ = "sentence t is **not** true when talking about t"

$T =$ Set of well-formed sentences

$$\begin{tikzcd}
T \times T \arrow[r, "f"] & \{0,1\} \arrow[d, "\neg"] \\
T \arrow[u, "\Delta"] \arrow[r, "g"'] & \{0,1\}
\end{tikzcd}$$

$g(t)$ = "sentence t is **not** true when talking about itself"

$T =$ Set of well-formed sentences

$$\begin{tikzcd} T \times T \arrow[r, "f"] & \{0,1\} \arrow[d, "\neg"] \\ T \arrow[u, "\Delta"] \arrow[r, "g"'] & \{0,1\} \end{tikzcd}$$

$g(t)$ = "sentence t is **not** true when talking about itself"

$t =$ "This sentence is not true when talking about itself"

$T =$ Set of well-formed sentences

$$\begin{tikzcd}
T \times T \arrow[r, "f"] & \{0,1\} \arrow[d, "\neg"] \\
T \arrow[u, "\Delta"] \arrow[r, "g"'] & \{0,1\}
\end{tikzcd}$$

$g(t)$ = "sentence t is **not** true when talking about itself"

$t =$ "This sentence is false"

Let T be the set of Turing Machines.

$$\begin{tikzcd}
T \times T \arrow[r, "f"] & \{0,1\} \arrow[d, "\neg"] \\
T \arrow[u, "\Delta"] \arrow[r, "g"'] & \{0,1\}
\end{tikzcd}$$

Let T be the set of Turing Machines.

$$\begin{tikzcd}
T \times T \arrow[r, "f"] & \{0,1\} \arrow[d, "\neg"] \\
T \arrow[u, "\Delta"] \arrow[r, "g"'] & \{0,1\}
\end{tikzcd}$$

Consider the pair of Turing Machines (M, M').

Let T be the set of Turing Machines.

$$\begin{tikzcd}
T \times T \arrow[r, "f"] & \{0,1\} \arrow[d, "\neg"] \\
T \arrow[u, "\Delta"] \arrow[r, "g"'] & \{0,1\}
\end{tikzcd}$$

Consider the pair of Turing Machines (M, M').

The description of M' is some string w.

Let T be the set of Turing Machines.

$$\begin{tikzcd}
T \times T \arrow[r, "f"] & \{0,1\} \arrow[d, "\neg"] \\
T \arrow[u, "\Delta"] \arrow[r, "g"'] & \{0,1\}
\end{tikzcd}$$

Consider the pair of Turing Machines (M, M').

We denote the description of M' by $\langle M' \rangle$.

Let T be the set of Turing Machines.

$$\begin{tikzcd}
T \times T \arrow[r, "f"] & \{0,1\} \arrow[d, "\neg"] \\
T \arrow[u, "\Delta"] \arrow[r, "g"'] & \{0,1\}
\end{tikzcd}$$

Consider the pair of Turing Machines (M, M').

We denote the description of M' by $\langle M' \rangle$.

Let $f(M, M')$ evaluate the machine M on string $\langle M' \rangle$.

Let T be the set of Turing Machines.

$$\begin{tikzcd}
T \times T \arrow[r, "f"] & \{0,1\} \arrow[d, "\neg"] \\
T \arrow[u, "\Delta"] \arrow[r, "g"'] & \{0,1\}
\end{tikzcd}$$

Consider the pair of Turing Machines (M, M').

$$f(M, M') = 1 \iff (M, \langle M' \rangle) \in A_{\mathrm{TM}}$$

Let T be the set of Turing Machines.

$$\begin{tikzcd}
T \times T \arrow[r, "f"] & \{0,1\} \arrow[d, "\neg"] \\
T \arrow[u, "\Delta"] \arrow[r, "g"'] & \{0,1\}
\end{tikzcd}$$

For sake of contradiction, suppose:

$$\overline{A_{\mathrm{TM}}} \in \mathrm{RE}(\Sigma).$$

Then there exists a machine R such that $\mathscr{L}(R) = \overline{A_{\mathrm{TM}}}$.

$$\overline{A_{\mathrm{TM}}} = \{(\langle M \rangle, w) : M \text{ does NOT accept } w\}$$

So R behaves like

- If M does not accept w, then R accepts $(\langle M \rangle, w)$.
- If M accepts w, then R may reject or loop indefinitely on $(\langle M \rangle, w)$.

Now consider another Turing Machine D
such that on input $\langle M \rangle$, run R on $(\langle M \rangle, \langle M \rangle)$.

$$\overline{A_{\mathrm{TM}}} = \{(\langle M \rangle, w) : M \text{ does NOT accept } w\}$$

So R behaves like

- If M does not accept w, then R accepts $(\langle M \rangle, w)$.
- If M accepts w, then R may reject or loop indefinitely on $(\langle M \rangle, w)$.

$$D \text{ accepts } \langle M \rangle \iff R \text{ accepts } (\langle M \rangle, \langle M \rangle)$$

$$\overline{A_{\mathrm{TM}}} = \{(\langle M \rangle, w) : M \text{ does NOT accept } w\}$$

So R behaves like

- If M does not accept w, then R accepts $(\langle M \rangle, w)$.
- If M accepts w, then R may reject or loop indefinitely on $(\langle M \rangle, w)$.

$$D \text{ accepts } \langle M \rangle \iff M \text{ does not accept } \langle M \rangle$$

Let T be the set of Turing Machines.

$$\begin{tikzcd}
T \times T \arrow[r, "f"] & \{0,1\} \arrow[d, "\neg"] \\
T \arrow[u, "\Delta"] \arrow[r, "g"'] & \{0,1\}
\end{tikzcd}$$

D accepts $\langle M \rangle \iff M$ does not accept $\langle M \rangle$

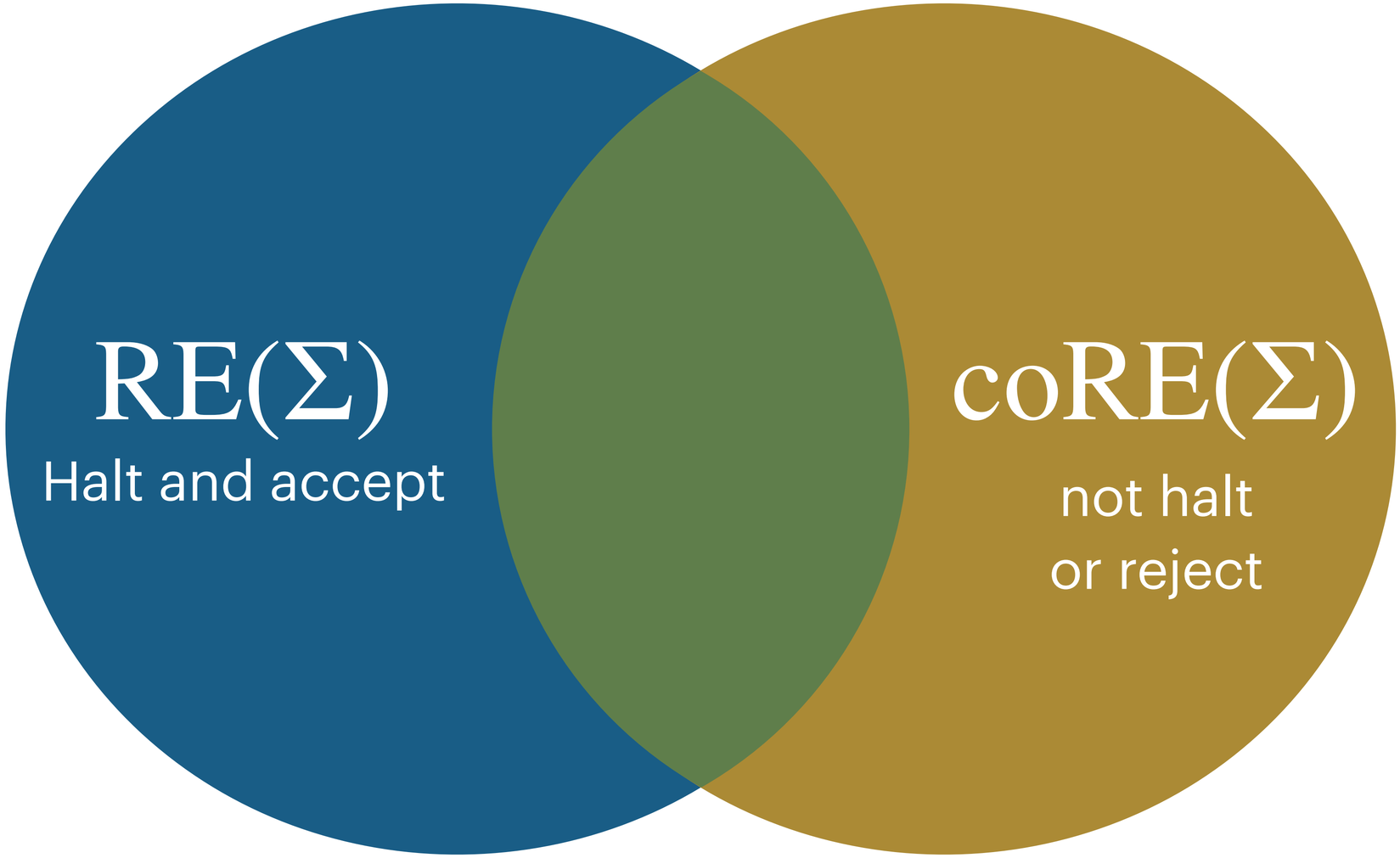
Let T be the set of Turing Machines.

$$\begin{tikzcd}
T \times T \arrow[r, "f"] & \{0,1\} \arrow[d, "\neg"] \\
T \arrow[u, "\Delta"] \arrow[r, "g"'] & \{0,1\}
\end{tikzcd}$$

D accepts $\langle M \rangle \iff M$ does not accept $\langle M \rangle$

Does D accept $\langle D \rangle$?

RE is not closed under complement.



$A_{\mathrm{TM}} \in \mathrm{RE}\,(\Sigma)$, but $A_{\mathrm{TM}} \notin \mathrm{coRE}\,(\Sigma)$.

Decidability

A language $L \subseteq \Sigma^*$ is **recursive** if

$$L \in \mathrm{RE}(\Sigma) \cap \mathrm{coRE}(\Sigma) = R(\Sigma).$$

Given $L \in R(\Sigma)$, there exists a **total** Turing Machine accepting L.

Decidability

A language $L \subseteq \Sigma^*$ is **recursive** if

$$L \in \mathrm{RE}(\Sigma) \cap \mathrm{coRE}(\Sigma) = R(\Sigma).$$

Given $L \in R(\Sigma)$, there exists a **total** Turing Machine accepting L.

There exists M_1 that accepts L.

There exists M_2 that accepts $\overline{L}$.

Decidability

A language $L \subseteq \Sigma^*$ is **recursive** if

$$L \in \mathrm{RE}(\Sigma) \cap \mathrm{coRE}(\Sigma) = R(\Sigma).$$

Given $L \in R(\Sigma)$, there exists a **total** Turing Machine accepting L.

By dovetailing M_1 and M_2, we get a total Turing Machine for L.

Shortlex Enumerators

An enumerator is a Turing machine

$$E = (Q, \Sigma, \#, \vdash, \delta, s)$$

$$\mathscr{L}(E) = \{w : E \text{ eventually prints } w\}$$

The shortlex order on Σ^* first compares word by length, and then in lexicographic (dictionary) order. For example:

$$a < b < aa < ab < ba < bb$$

Shortlex Enumerators

$L \in \mathrm{R}(\Sigma) \Rightarrow \exists$ shortlex enumerator $E \,.\, L = \mathscr{L}(E)$

Consider shortlex enumeration over Σ^*: $w_1, w_2, w_3, \ldots$

Run M on $w_1, w_2, w_3, \ldots$.

If accepted, enumerate.

Else, skip.

What about the converse?

All *decision algorithms* check acceptance in a recursive language.

Turing Machines, enumerators, programming languages, 2-stack automata, 2-counter automata, queue automata, etc...

all check acceptance in recursively enumerable languages.

Recursively enumerable languages are
not closed under complement.

Universality of CFL

Consider the language of universal CFGs:

$$\mathrm{ALL}_{\mathrm{CFL}} = \{G : \mathscr{L}(G) = \Sigma^*\}.$$

Is $\mathrm{ALL}_{\mathrm{CFG}} \in \mathrm{R}(\Sigma)$?

Refutation is easy. Proof is hard!

Universality of CFL

Consider the language of universal CFGs:

$$\mathrm{ALL}_{\mathrm{CFL}} = \{G : \mathscr{L}(G) = \Sigma^*\}.$$

We will use the fact that $\overline{A_{\mathrm{TM}}} \notin \mathrm{RE}(\Sigma)$.

Universality of CFL

Let M be a Turing Machine.

An accepting computation of M on w is $C_0\#C_1\#\cdots\#C_k$ where C_0 is the start configuration of M on w.

Universality of CFL

Let M be a Turing Machine.

An accepting computation of M on w is $C_0\#C_1\#\cdots\#C_k$ where C_0 is the start configuration of M on w.

Let $\mathrm{VAL}_{M,w} = \{x : x$ is a valid accepting computation of M on $w\}$

Universality of CFL

Let M be a Turing Machine.

An accepting computation of M on w is $C_0\#C_1\#\cdots\#C_k$ where C_0 is the start configuration of M on w.

Let $\mathrm{VAL}_{M,w} = \{x : x$ is a valid accepting computation of M on $w\}$

For all Turing Machines M, and words w, prove that $\overline{\mathrm{VAL}_{M,w}}$ is context-free.